

Programming I/O in Linux

I. Objectives

- Students understand the concept of files, different types of files, file attributes, basic system calls when working with file and directory.
- Students can distinguish the difference when using system calls and library functions to performing I/O.
- Students can using system calls and library functions for required tasks on handling I/O.

II. Basic theory

1. Working with files on Linux system

1.1. File and attributes

In Linux, a file is essentially any named collection of data stored on a disk. Different types of files include:

- **Regular file:** They store actual data and can be read, written, or executed based on permissions. (Ex: text files, binary files...)
- **Directory:** Directories are special files that serve as containers for other files and subdirectories. They map file names to their corresponding inodes (the file system's data structures that store file attributes and location data).
- **Device files:** a special file that acts as an interface to hardware or virtual devices. Instead of storing data, these files allow software to communicate with the corresponding device drivers in the kernel, which then interacts with the actual hardware or virtual device. These files, usually found in the /dev directory.
- **FIFO (pipe):** FIFOs provide a way for processes to communicate with each other. They allow one process to write data which another process reads in a first-in-first-out manner.
- **Sockets:** Sockets are similar to FIFOs but are more versatile. They allow bidirectional communication between processes, potentially over a network.

In Linux, “everything is a file” is a foundational principle that means nearly every object you interact with in the system is represented as a file and accessed using file-like interfaces. This means that whether you’re dealing with a regular file, a directory, a device file, a pipe, or even a network socket, the same set of system calls—such as open, read, write, and close—can be used to perform input/output operations.

Each file in Linux not only contains data but also holds numerous descriptive attributes stored in its inode. The struct **stat** structure is used to hold the file attributes information retrieved from the inode. Some important fields in struct **stat** include:

```

struct stat {
    dev_t    st_dev;        /* IDs of device on which file resides */
    ino_t    st_ino;        /* I-node number of file */
    mode_t    st_mode;      /* File type and permissions */
    nlink_t   st_nlink;     /* Number of (hard) links to file */
    uid_t     st_uid;       /* User ID of file owner */
    gid_t     st_gid;       /* Group ID of file owner */
    dev_t     st_rdev;      /* IDs for device special files */
    off_t     st_size;      /* Total file size (bytes) */
    blksize_t st_blksize;   /* Optimal block size for I/O (bytes) */
    blkcnt_t  st_blocks;    /* Number of (512B) blocks allocated */
    time_t    st_atime;     /* Time of last file access */
    time_t    st_mtime;     /* Time of last file modification */
    time_t    st_ctime;     /* Time of last status change */
};

```

Fields	Description																																																					
st_mode	This is a bit mask serving the dual purpose of identifying the file type and specifying the file permissions. The bits of this field are laid out as shown below: ← File type → ← Permissions → <table><tr><td></td><td></td><td></td><td></td><td>U</td><td>G</td><td>T</td><td>R</td><td>W</td><td>X</td><td>R</td><td>W</td><td>X</td><td>R</td><td>W</td><td>X</td></tr><tr><td colspan="4"></td><td colspan="3">User</td><td colspan="3">Group</td><td colspan="3">Other</td></tr></table> The file type can be extracted from this field by ANDing (&) with the constant S_IFMT. The resulting value can then be compared with a range of constants to determine the file type, like so <pre>if ((statbuf.st_mode & S_IFMT) == S_IFREG) printf("regular file\n");</pre> Macros for checking file types in the st_mode field of the stat structure: <table><tr><th>Constant</th><th>Test macro</th><th>File type</th></tr><tr><td>S_IFREG</td><td>S_ISREG()</td><td>Regular file</td></tr><tr><td>S_IFDIR</td><td>S_ISDIR()</td><td>Directory</td></tr><tr><td>S_IFCHR</td><td>S_ISCHR()</td><td>Character device</td></tr><tr><td>S_IFBLK</td><td>S_ISBLK()</td><td>Block device</td></tr><tr><td>S_IFIFO</td><td>S_ISFIFO()</td><td>FIFO or pipe</td></tr><tr><td>S_IFSOCK</td><td>S_ISSOCK()</td><td>Socket</td></tr><tr><td>S_IFLNK</td><td>S_ISLNK()</td><td>Symbolic link</td></tr></table>					U	G	T	R	W	X	R	W	X	R	W	X					User			Group			Other			Constant	Test macro	File type	S_IFREG	S_ISREG()	Regular file	S_IFDIR	S_ISDIR()	Directory	S_IFCHR	S_ISCHR()	Character device	S_IFBLK	S_ISBLK()	Block device	S_IFIFO	S_ISFIFO()	FIFO or pipe	S_IFSOCK	S_ISSOCK()	Socket	S_IFLNK	S_ISLNK()	Symbolic link
				U	G	T	R	W	X	R	W	X	R	W	X																																							
				User			Group			Other																																												
Constant	Test macro	File type																																																				
S_IFREG	S_ISREG()	Regular file																																																				
S_IFDIR	S_ISDIR()	Directory																																																				
S_IFCHR	S_ISCHR()	Character device																																																				
S_IFBLK	S_ISBLK()	Block device																																																				
S_IFIFO	S_ISFIFO()	FIFO or pipe																																																				
S_IFSOCK	S_ISSOCK()	Socket																																																				
S_IFLNK	S_ISLNK()	Symbolic link																																																				

st_ino	The field contains the i-node number of the file
st_dev	The dev_t type records the major and minor IDs of a device
<i>The combination of st_dev and st_ino uniquely identifies a file across all file systems</i>	
st_nlink	The number of hard links pointing to the file.
st_uid, st_gid	The st_uid and st_gid fields identify, respectively, the owner (user ID) and group (group ID) to which the file belongs.
st_size	The file size in bytes
st_atime, st_mtime, st_ctime	The times of last access, modification of content, and time when the inode's status was last changed (not just content modifications).

To retrieve file attribute information, there are three main system calls used:

```
#include <sys/stat.h>

int stat(const char *pathname, struct stat *statbuf);
int lstat(const char *pathname, struct stat *statbuf);
int fstat(int fd, struct stat *statbuf);
```

stat() system call:

Purpose: return information about a file specified in **pathname**, in the buffer pointed to by **statbuf**. If the file is a symbolic link, this function follows the link and returns information about the target file.

Example

```
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    struct stat sb;
    if (stat("path/to/file", &sb) == -1) {
        perror("stat");
        exit(EXIT_FAILURE);
    }
    printf("File size: %lld bytes\n", (long long) sb.st_size);
    return 0;
}
```

```
}
```

lstat() system call

Purpose: Similar to [stat\(\)](#), but it does not follow symbolic links. If the provided path is a symbolic link, the function returns information about the link itself rather than the target file.

fstat() system call

Purpose: Retrieves file information based on a file descriptor (fd) obtained from an [open\(\)](#) call. This is useful when the file is already open and you want to get its attributes without referring to its path again.

1.2 Basic file operations

Basic Linux system call handle file operations are: *open()*, *read()*, *write()*, *close()*.

Open a file: *open()* system call

```
#include <sys/stat.h>
#include <fcntl.h>
int open (const char *pathname, int flags, ... /* mode_t mode */);
Returns file descriptor on success, or -1 on error
```

The file to be opened is identified by the **pathname** argument. If **pathname** is a symbolic link, it is dereferenced. On success, *open()* returns a file descriptor that is used to refer to the file in subsequent system calls and uses the lowest-numbered unused file descriptor for the process. If an error occurs, *open()* returns -1 and **errno** is set accordingly.

The **flags** argument is a bit mask that are divided into the following groups: file access mode flags, file creation flags and open file status flags (three parts in table below).

Table 4-3: Values for the *flags* argument of *open()*

Flag	Purpose	SUS?
O_RDONLY	Open for reading only	v3
O_WRONLY	Open for writing only	v3
O_RDWR	Open for reading and writing	v3
O_CLOEXEC	Set the close-on-exec flag (since Linux 2.6.23)	v4
O_CREAT	Create file if it doesn't already exist	v3
O_DIRECT	File I/O bypasses buffer cache	
O_DIRECTORY	Fail if <i>pathname</i> is not a directory	v4
O_EXCL	With O_CREAT: create file exclusively	v3
O_LARGEFILE	Used on 32-bit systems to open large files	
O_NOATIME	Don't update file last access time on <i>read()</i> (since Linux 2.6.3)	
O_NOCTTY	Don't let <i>pathname</i> become the controlling terminal	v3
O_NOFOLLOW	Don't dereference symbolic links	v4
O_TRUNC	Truncate existing file to zero length	v3
O_APPEND	Writes are always appended to end of file	v3
O_ASYNC	Generate a signal when I/O is possible	
O_DSYNC	Provide synchronized I/O data integrity (since Linux 2.6.33)	v3
O_NONBLOCK	Open in nonblocking mode	v3
O_SYNC	Make file writes synchronous	v3

When *open()* is used to create a new file, the **mode** bit-mask argument specifies the permissions to be placed on the file.

If an error occurs while trying to open the file, *open()* returns -1, and **errno** identifies the cause of the error. The following are some possible errors that can occur:

Error	Description
EACCES	The file permissions don't allow the calling process to open the file in the mode specified by flags. Alternatively, because of directory permissions, the file could not be accessed, or the file did not exist and could not be created.
EISDIR	The specified file is a directory, and the caller attempted to open it for writing. This isn't allowed
EMFILE	The process resource limit on the number of open file descriptors has been reached
ENFILE	The system-wide limit on the number of open files has been reached
ENOENT	The specified file doesn't exist, and O_CREAT was not specified, or O_CREAT was specified, and one of the directories in pathname doesn't exist or is a symbolic link pointing to a nonexistent pathname (a dangling link).
EROFS	The specified file is on a read-only file system and the caller tried to open it for writing.
ETXTBSY	The specified file is an executable file (a program) that is currently executing. It is not permitted to modify (i.e., open for writing) the executable file associated with a running program.

Example:

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int fd = open("example.txt", O_RDWR);
    if (fd < 0) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    printf("File opened successfully with descriptor: %d\n", fd);
    // File remains open for further operations.
    return 0;
}
```

Read from a file: read() system call

```
#include <unistd.h>
ssize_t read(int fd, void *buffer, size_t count);
```

Returns number of bytes read, 0 on EOF, or -1 on error

The *read()* system call reads data from the open file referred to by the descriptor **fd**. The **count** argument specifies the maximum number of bytes to read. (The `size_t` data type is an unsigned integer type.) The **buffer** argument supplies the address of the memory buffer into which the input data is to be placed. This buffer must be at least **count** bytes long.

A successful call to *read()* returns the number of bytes actually read, or 0 if end-of-file is encountered. On error, the usual -1 is returned. The `ssize_t` data type is a signed integer type used to hold a byte count or a -1 error indication.

A call to *read()* may read less than the requested number of bytes. For a regular file, the probable reason for this is that we were close to the end of the file. Note that *read()* doesn't place a terminating null byte at the end of the string that *printf()* is being asked to print. If a terminating null byte is required at the end of the input buffer, we must put it there explicitly.

Example:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    //....Open file descriptor
    char buffer[128];
    ssize_t bytesRead = read(fd, buffer, sizeof(buffer) - 1);
    if (bytesRead < 0) {
        perror("read");
        close(fd);
        exit(EXIT_FAILURE);
    }
    buffer[bytesRead] = '\0'; // Null-terminate the string.
    printf("Read %zd bytes: %s\n", bytesRead, buffer);
    //....Close file descriptor
    return 0;
}
```

Writing to a file: **write()**

The *write()* system call writes data to an open file.

```
#include <unistd.h>
ssize_t write(int fd, void *buffer, size_t count);
Returns number of bytes written, or -1 on error
```

The arguments to *write()* are similar to those for *read()*: **buffer** is the address of the data to be written; **count** is the number of bytes to write from **buffer**; and **fd** is a file descriptor referring to the file to which data is to be written.

On success, *write()* returns the number of bytes actually written; this may be less than **count**. For a disk file, possible reasons for such a partial write are that the disk was filled or that the process resource limit on file sizes was reached.

Example:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    //.... Open file descriptor

    const char *message = "Hello, world!\n";
    ssize_t bytesWritten = write(fd, message, strlen(message));
    if (bytesWritten < 0) {
        perror("write");
        close(fd);
        exit(EXIT_FAILURE);
    }
    printf("Wrote %zd bytes to output.txt\n", bytesWritten);

    //.... Close file descriptor
    return 0;
}
```

Closing a file: *close()* system call

```
#include <unistd.h> int
close(int fd);
Returns 0 on success, or -1 on error
```

The *close()* system call closes an open file descriptor, freeing it for subsequent reuse by the process. When a process terminates, all of its open file descriptors are automatically closed.

It is usually good practice to close unneeded file descriptors explicitly, since this makes our code more readable and reliable in the face of subsequent modifications. Furthermore, file descriptors are a consumable resource, so failure to close a file descriptor could result in a process running out of descriptors.

Example:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {

    //.... Open file descriptor

    // ... Perform operations on the file descriptor ...

    if (close(fd) < 0) {
        perror("close");
        exit(EXIT_FAILURE);
    }
    printf("File closed successfully.\n");
    return 0;
}
```

Changing the file offset: `lseek()` system call

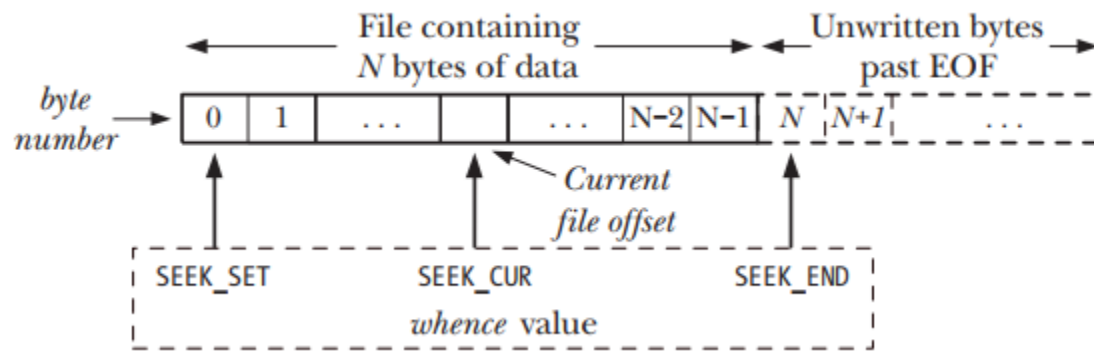
```
#include <unistd.h>
off_t lseek(int fd, off_t offset, int whence);
Returns new file offset if successful, or -1 on error
```

For each open file, the kernel records a file offset, sometimes also called the readwrite offset or pointer. This is the location in the file at which the next `read()` or `write()` will commence. The file offset is expressed as an ordinal byte position relative to the start of the file. The first byte of the file is at offset 0.

The `lseek()` system call adjusts the file offset of the open file referred to by the file descriptor **fd**, according to the values specified in **offset** and **whence**.

The **offset** argument specifies a value in bytes. The **whence** argument indicates the base point from which offset is to be interpreted, and is one of the following values:

- `SEEK_SET`: The file offset is set **offset** bytes from the beginning of the file.
- `SEEK_CUR`: The file offset is adjusted by **offset** bytes relative to the current file offset.
- `SEEK_END`: The file offset is set to the size of the file plus **offset**. In other words, offset is interpreted with respect to the next byte after the last byte of the file.



Example:

```
lseek(fd, 0, SEEK_SET); /* Start of file */
lseek(fd, 0, SEEK_END); /* Next byte after the end of the file */
lseek(fd, -1, SEEK_END); /* Last byte of file */
lseek(fd, -10, SEEK_CUR); /* Ten bytes prior to current location */
lseek(fd, 10000, SEEK_END); /* 10001 bytes past last byte of file */
```

Linux system calls and standard C library

System calls are the primary interface between user applications and the Linux kernel. They allow programs to request services from the kernel, such as file I/O, process control, and networking. System call perform low-level, fundamental operations and provide a minimal and often less user-friendly interface.

The C standard library (e.g., glibc) builds on top of system calls, providing a more user-friendly, high-level interface and often include extra features. These library functions simplify programming by abstracting many of the low-level details of direct system call usage.

Below is a table summarizing the syntax and usage of several common standard C library functions for I/O programming:

Function	Syntax	Usage
printf	int printf(const char *format, ...);	Formats and prints output to the standard output (stdout).
scanf	int scanf(const char *format, ...);	Reads formatted input from the standard input (stdin).
fprintf	int fprintf(FILE *stream, const char *format, ...);	Formats and writes output to a specified file stream.
fscanf	int fscanf(FILE *stream, const char *format, ...);	Reads formatted input from a specified file stream.
fopen	FILE *fopen(const char *filename, const char *mode);	Opens a file with a given mode (e.g., "r", "w", "a") and returns a file pointer.
fclose	int fclose(FILE *stream);	Closes an open file stream.
fread	size_t fread(void *ptr, size_t size, size_t count, FILE *stream);	Reads data from a file into a memory block.

fwrite	size_t fwrite(const void *ptr, size_t size, size_t count, FILE *stream);	Writes data from a memory block to a file.
fgets	char *fgets(char *str, int n, FILE *stream);	Reads a line or string from a file, up to n-1 characters (including the newline if encountered).
fputs	int fputs(const char *str, FILE *stream);	Writes a string to a file without automatically appending a newline.
fgetc	int fgetc(FILE *stream);	Reads a single character from the specified stream and returns it as an unsigned char cast to an int; returns EOF on end-of-file or error.
fputc	int fputc(int c, FILE *stream);	Writes a single character (passed as an int) to the specified stream; returns the character written or EOF on error.
freopen	FILE *freopen(const char *filename, const char *mode, FILE *stream);	Reopens an existing stream with a new file and mode, often used to redirect standard streams like stdin, stdout, or stderr to a file.

2. Working with Directory

2.1. The `struct dirent` Structure

Directories in Linux are special files that contain a list of entries representing files or subdirectories. To operate on directories, the system provides several basic system calls and an important data structure called `struct dirent`.

The `struct dirent` structure holds information about each entry within a directory:

d_ino	The inode number of the entry
d_name	The name of the entry (either a file or a directory)
d_type	(If available) The type of the entry, such as DT_REG for regular files, DT_DIR for directories, DT_LNK for symbolic links...

(Note: Not all systems support this field.)

2.2. Directory Handling System calls

The `opendir()` system call

The `opendir()` function opens a directory and returns a handle that can be used to refer to the directory in later calls. This function opens the directory specified by `dirpath` and returns a pointer to a structure of type DIR.

```
#include <dirent.h>
DIR *opendir(const char *dirpath);
```

Example

```

#include <dirent.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    DIR *dp = opendir("path/to/directory");
    if (dp == NULL) {
        perror("opendir");
        exit(EXIT_FAILURE);
    }
    //Perform directory operations here
    closedir(dp);
    return 0;
}

```

The `readdir()` system call

The `readdir()` function reads successive entries from a directory stream.

```

#include <dirent.h>
struct dirent *readdir(DIR *dirp);
Returns pointer to a statically allocated structure describing next directory entry, or NULL on end-of-directory or error

```

Each call to `readdir()` reads the next directory from the directory stream referred to by `dirp` and returns a pointer to a statically allocated structure of type `dirent`, containing the following information about the entry:

```

struct dirent {
    ino_t d_ino;
    char d_name[];
};

```

Example

```

#include <dirent.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    //.....open directory
    struct dirent *entry;
    while ((entry = readdir(dp)) != NULL) {
        printf("Found entry: %s\n", entry->d_name);
    }
    //.....close directory
    return 0;
}

```

The `mkdir()` system call

```
#include <sys/stat.h>
int mkdir(const char *pathname, mode_t mode);
```

Creates a new directory with specified access permissions.

The `pathname` argument specifies the pathname of the new directory. This path name may be relative or absolute. If a file with this pathname already exists, then the call fails with the error `EEXIST`.

The `mode` argument specifies the permissions for the new directory.

For example, permission 0755 allows the owner full access (read, write, execute), while the group and others have read and execute permissions only.

Example

```
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    if (mkdir("path/to/new_directory", 0755) == -1) {
        perror("mkdir");
        exit(EXIT_FAILURE);
    }
    printf("Directory created successfully!\n");
    return 0;
}
```

The `rmdir()` system call

```
#include <unistd.h>
int rmdir(const char *pathname);
Returns 0 on success, or -1 on error
```

The `rmdir()` system call removes the directory specified in `pathname`, which may be an absolute or a relative pathname.

In order for `rmdir()` to succeed, the directory must be empty. If the final component of `pathname` is a symbolic link, it is not dereferenced; instead, the error `ENOTDIR` results.

2.3. Some important considerations when working with directories

- Always check the return values of functions like `opendir()`, `readdir()`, and `mkdir()` to detect errors (for instance, due to insufficient permissions or non-existent directories) and handle them appropriately.

- After opening a directory with `opendir()`, remember to call `closedir()` to free up system resources.
- When traversing a directory, entries such as `"."` (the current directory) and `".."` (the parent directory) are often present. You might need to handle these separately if you do not want them to appear in your results.

3. Compiling and run C program

To compile a C program, use the **gcc** compiler:

```
gcc <source file> -o <executable file>
```

Example: `gcc Hello.c -o Hello`

`cc` → Calls the GCC compiler

`Hello.c` → The source code file

`-o Hello` → Specifies the output executable name (hello)

Common GCC Compilation Flags

Flag	Purpose
<code>-o <file></code>	Name the output executable
<code>-Wall</code>	Show all warnings
<code>-g</code>	Include debugging information (useful for gdb)

To run compiled program: `./Hello`